Chaos and Fractals: Patterns from Simple Rules

Hartwell Markham, Rob Tompkins, Rekik Ziku

November 12, 2026

Abstract

This project explores how simple repeated rules can produce intricate patterns, connecting the study of chaos with the geometry of fractals. Chaos concerns systems whose behavior can become difficult to predict because small differences in their starting conditions grow over time, while fractals describe shapes and sets with detail at increasingly small scales. These ideas often meet, but fractal structure does not by itself imply chaotic behavior. Using Chapter 11 of Strogatz's *Nonlinear Dynamics and Chaos* as a foundation, we will consider how mathematicians describe the size and complexity of such patterns through countability, uncountability, and fractal dimension. The Cantor set provides a central example: removing intervals repeatedly leaves an uncountable collection of points with zero total length. Box dimension offers another way to understand its size by tracking how many small boxes are needed to cover it as the scale becomes finer.

Two examples will connect these abstract ideas to the emergence of recognizable patterns: the chaos game and the Devil's staircase. In the familiar chaos game, repeatedly moving partway toward a randomly chosen vertex of a triangle produces the Sierpiński triangle, showing how random choices can reveal an organized geometric structure. Barnsley and Vince's *The chaos game on a general iterated function system* explains why this approach can recover an attracting set under broad conditions, extending beyond the usual setting in which every rule shrinks distances. Per Bak's *The Devil's staircase* examines a different source of intricate organization: systems with competing rhythms that synchronize over ranges of conditions, creating a hierarchy of plateaus separated by a Cantor-like set. Together, these examples frame the project around a shared question: how does repeated evolution produce structure across scales? By comparing a fractal generated by random iteration with fractal organization among the conditions for synchronization, we will use visual examples and accessible geometric ideas to connect unpredictability, order, and the different meanings of mathematical size.

1 Random iteration: the circle attractor

We've used both python and matlab to reproduce Example 1 of Barnsley and Vince [2], we take the state space to be the unit circle and at each step independently choose either the identity or a rotation, each with probability 1/2. We use the irrational rotation angle $2\pi(\sqrt{5}-1)/2$ and start at $(1,0)$. Both maps preserve distances, so this example tests the chaos game beyond the familiar contracting maps that generate the Sierpiński triangle.

Write the circle as $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ and let $\alpha = (\sqrt{5}-1)/2$. From phase t_0, the set of all n-step images is $\{t_0 + j\alpha : 0 \le j \le n\}$ modulo one. The forward orbit of an irrational rotation is dense, and compactness implies that some finite prefix is an ε-net for each $\varepsilon > 0$. For any nonempty compact initial set B on the circle, the n-step images of B contain the images of a single point of B and lie on the circle, so their Hausdorff limit is the circle as well.

In the random experiment let S_n count rotations among the first n choices. The phase at step n is $t_0 + \alpha S_n$ modulo one, and the strong law gives $S_n/n \to 1/2$ almost surely. Because the count

increases by zero or one, every tail visits all rotation counts from its starting count onward. Thus every tail is dense almost surely, even though two starting phases driven by the same choices stay the same distance apart. Attraction describes the coverage by sets of images here; individual trajectories need not approach one another.

For a finite set of sampled points, let g_N denote the largest gap between successive distinct phases, measured in turns and including the wraparound gap. The point farthest from the sample is at the midpoint of that gap, so the Euclidean Hausdorff distance to the circle is

$$d_H = 2\sin(\pi g_N/2).$$

The midpoint of the largest gap is at angular distance πg_N from its nearest sample, and the corresponding chord has length $2\sin(\pi g_N/2)$. The reverse directed distance is zero because every sample point lies on the circle. If there are M distinct phases, the gaps sum to one, so $g_N \geq 1/M$. Thus $2\sin(\pi/(2M))$ is the best possible coverage error for M samples, attained by equal spacing.

The experiment uses a fixed random seed and measures cumulative coverage at powers of two. The error drops from 1.129 after 8 iterates to 0.00230 after 4096. Figure 1 reproduces the qualitative finding that a single random orbit draws the circle and adds a numerical measure of how well the sample covers it. The finite prefix calculation illustrates coverage; the almost sure statement about infinite tails comes from the argument above and the theorem, rather than the plot.

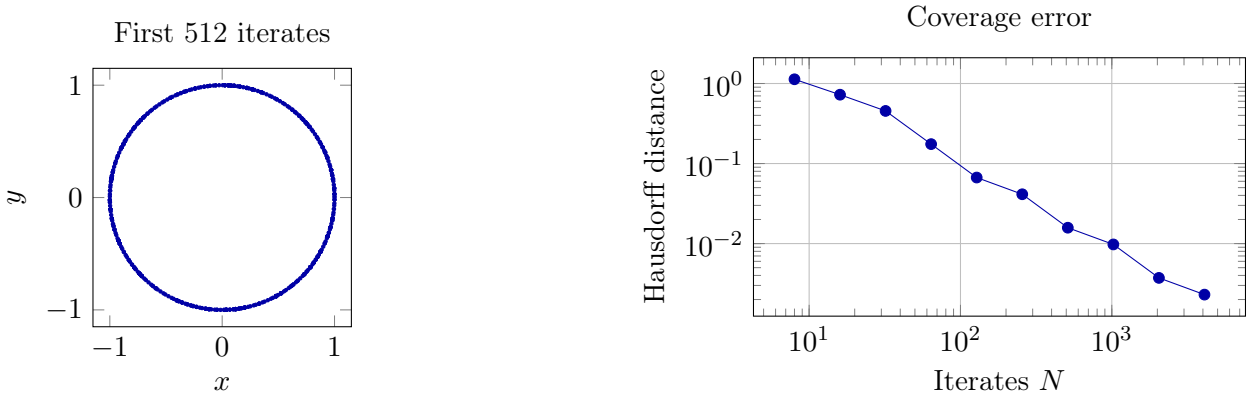


Figure 1: The noncontractive chaos game on the unit circle. Repeated points do not improve coverage; newly visited phases close the gaps.

2 Frequency locking: the critical Devil's staircase

To reproduce the winding number staircase in Figure 5(c) of Bak [3], we iterate the lift of the sine circle map,

$$x_{n+1} = x_n + \Omega + \frac{K}{2\pi}\sin(2\pi x_n).$$

The bias Ω is the uncoupled advance per step, and K sets the strength of the periodic term. At the critical value $K = 1$, we estimate the winding number by averaging the unwrapped increments after a transient. The implementation keeps the phase modulo one for evaluating the sine but accumulates the full increments. We use 3001 equally spaced values of Ω on $[0, 1]$, discard 4096 steps and average the next 16384, starting all runs at phase 0.123.

2.1 Existence of the winding number and mechanism of locking

Write F_Ω for the lift at $K = 1$. It is increasing because $F'_\Omega(x) = 1 + \cos(2\pi x) \geq 0$ and the zeros of the derivative are isolated. It satisfies $F_\Omega(x + 1) = F_\Omega(x) + 1$. Let $D_n(x) = F^n_\Omega(x) - x$, and let a_n and b_n be its minimum and maximum over one period. For $x \leq y \leq x + 1$, the difference $F^n_\Omega(y) - F^n_\Omega(x)$ lies in $[0, 1]$, so $b_n - a_n \leq 1$. The identity $D_{n+m}(x) = D_n(x) + D_m(F^n_\Omega(x))$ makes a_n superadditive and b_n subadditive. By the superadditive and subadditive limit lemmas, their normalized limits coincide, giving a common winding number W for every initial phase. Moreover, $a_n \leq nW \leq b_n$, so $|D_n(x)/n - W| \leq 1/n$ for any starting phase. For 16384 iterates this bounds the finite-window error by 6.11×10^{-5} in exact arithmetic. Increasing Ω increases the lift pointwise, so monotonicity of the iterates makes W nondecreasing.

If $F^q_\Omega(x_*) = x_* + p$, then $F^{mq}_\Omega(x_*) = x_* + mp$ for every positive integer m, giving $W = p/q$. A root of $F^q_\Omega(x) - x - p = 0$ persists as Ω varies locally whenever the cycle multiplier $\lambda = (F^q_\Omega)'(x_*)$ differs from one, by the implicit function theorem. By the chain rule this multiplier is the product of the q derivatives along the cycle, and $0 \leq \lambda < 1$ gives local attraction. At $\Omega = 1/2$, the lift sends 0 to $1/2$ and then to 1, giving $W = 1/2$ and $\lambda = (1 + \cos 0)(1 + \cos \pi) = 0$. The resulting persistence of this period-two orbit explains why the central step has positive width.

2.2 Comparison with the numerical sweep

Figure 2 shows broad plateaus at 0, 1/2 and 1, with smaller steps including 1/3 and 2/3. For example, the computed winding number is within 2×10^{-5} of 1/2 at grid points from 0.463333 to 0.536667. This is a grid-based estimate, not a determination of the exact plateau endpoints. The maximum change across the grid when doubling both the transient and averaging windows from half their final values was 5.51×10^{-5}. As a separate check, a fixed point of the lift satisfies $\Omega = -\sin(2\pi x)/(2\pi)$ at $K = 1$. Thus the zero-winding plateau on $[0, 1]$ extends to $1/(2\pi) \approx 0.159155$, consistent with the last detected grid point at 0.159000. The finite grid resolves only some of the plateaus and cannot establish that the full staircase occupies full measure.

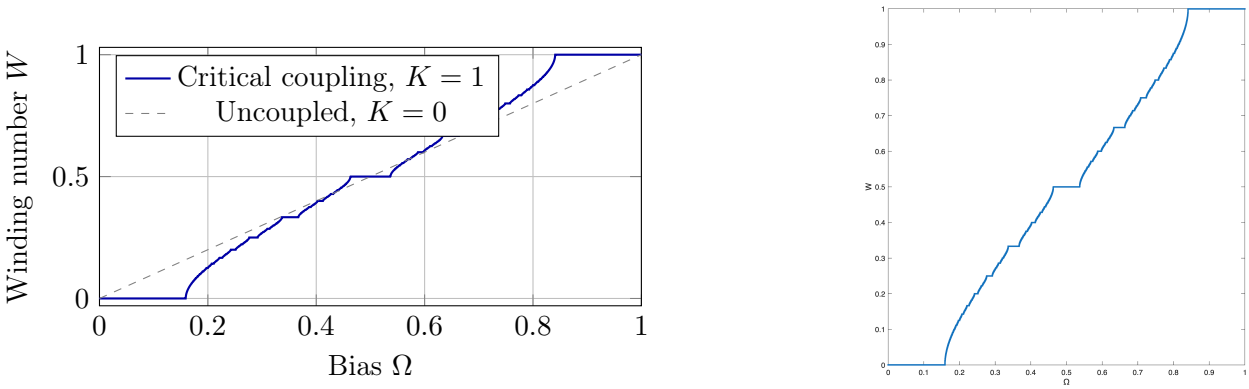


Figure 2: Numerical reproduction of Bak's critical winding number staircase. The Python/TikZ plot is on the left and the MATLAB export (`matlab_staircase.png`) is on the right. Horizontal segments indicate that the same average rotation persists over a range of biases.

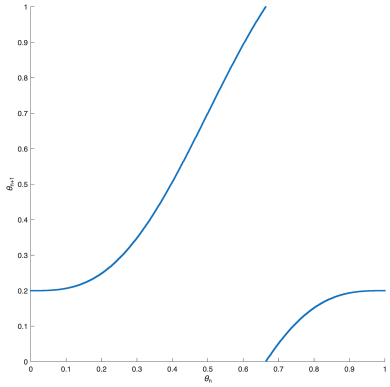


Figure 3: The MATLAB circle map (`matlab_circle_map.png`) at $\Omega = 0.2$ and $K = 1$. The graph shows one step modulo one for the map used to produce the staircase.

3 The connection through fractal dimension

Strogatz [1, Chapter 11] distinguishes counting points, measuring length, and counting boxes as the observation scale shrinks. For instance, the middle-thirds Cantor set has zero length and uncountably many points, yet at scale 3^{-m} it takes on the order of 2^m boxes to cover it, giving box dimension $\log 2/\log 3$. These are three different ways to describe the same set.

The circle experiment concerns the geometry of an attractor in state space. Its orbit has only countably many points, yet its closure is the entire circle, with box dimension one. The staircase experiment concerns geometry in parameter space: its fractal set lies on the bias axis, where the open locking intervals have been removed. Bak estimates its dimension near 0.87 by measuring how much of the bias axis remains uncovered after accounting for plateaus wider than a scale r and dividing the remaining length by r. To make the connection precise, let E be the closed set left after removing the open locking intervals and include the endpoints of $[0, 1]$. In the complete staircase, E has measure zero. If its complementary gaps have lengths ℓ_j, the length of the $r/2$-neighborhood of E inside $[0, 1]$ is $V(r) = \sum_j \min(\ell_j, r)$. Each gap wider than r contributes two strips of length $r/2$, and each shorter gap contributes its full length. Writing $L(r)$ for the number of gaps wider than r and $S(r)$ for their total length gives

$$\frac{V(r)}{r} = L(r) + \frac{1 - S(r)}{r}.$$

The ratio $V(r)/r$ is comparable up to constants to the number of mesh intervals of length r meeting E. The neighborhood lies in those intervals and their neighbors, and each occupied interval contains at least $r/2$ of neighborhood length. Bak fits the second term alone; the identity explains why it measures geometry at scale r without being literally the same as the mesh count.

The MATLAB experiment in Appendix A attempts the dimension calculation using widths detected on a finite grid. A fit on this grid alone does not verify Bak's dimension estimate. The shared lesson is that a finite orbit or a finite parameter scan shows structure at a chosen resolution; the limiting set explains what those observations are approaching.

Code and figures. The files can be found in the GitHub project directory. The source files are `chaos_game.py`, `devil_staircase.py`, and `circle_map_dimension.m`. The `plots/` directory contains the Python previews `chaos_game.png` and `devil_staircase.png`, and the MATLAB exports `matlab_circle_map.png`, `matlab_staircase.png`, and `matlab_dimension_fit.png`.

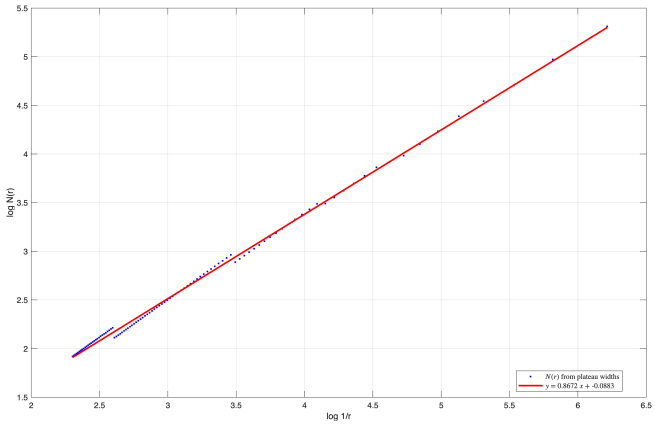


Figure 4: MATLAB fit of Bak's gap statistic (`matlab_dimension_fit.png`). The candidate dimension is the negative of the fitted slope.

References

[1] Steven H. Strogatz. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. 2nd ed., CRC Press, 2018. Chapter 11, "Fractals."

[2] Michael F. Barnsley and Andrew Vince. "The chaos game on a general iterated function system." *Ergodic Theory and Dynamical Systems*, 31(4), 1073–1079, 2011. Published online in 2010. DOI: `10.1017/S0143385710000428`.

[3] Per Bak. "The Devil's staircase." *Physics Today*, 39(12), 38–45, 1986.

A MATLAB circle-map and dimension experiment

The script plots the circle map modulo one, estimates the winding staircase from the lifted iterates, and fits Bak's gap statistic using plateaus detected by a tolerance on adjacent winding estimates. Its negative sine convention is equivalent to the positive sine convention in the paper under a shift of the phase by half a turn, which preserves the asymptotic winding number. Since the fit uses $\log r$ on the horizontal axis, the candidate dimension is the negative of the printed slope: $N(r) \propto r^{-d}$ gives $\log N(r) = \text{const} - d \log r$. The bias grid, finite iteration count, and flatness tolerance all affect which widths are detected, so the fitted slope is a finite-resolution estimate.

Listing 1: MATLAB script for the circle map, winding staircase, and gap statistic.

```
clear all; close all;
plotDir = fullfile(fileparts(mfilename('fullpath')), 'plots');
if ~isfolder(plotDir)
    mkdir(plotDir);
end
%% circle map
Om = 0.2; % omega
K = 1;
th = linspace(0,1,1000);
figure;
hold on
plot(th, mod(th + Om - (K/(2*pi))*sin(2*pi*th), 1),'.');
xlabel('\theta_n');
ylabel('\theta_{n+1}');
axis square
exportgraphics(gcf, fullfile(plotDir, 'matlab_circle_map.png'), '
   Resolution', 300);

%% staircase

Omega = linspace(0,1,20000);
N = 14000; % iterations
theta = zeros(size(Omega));
for k = 1:N
    theta = theta + Omega - (K/(2*pi))*sin(2*pi*theta);
end
W = theta / N;

figure;
plot(Omega, W, '.')
xlabel('\Omega');
ylabel('W');
axis square
exportgraphics(gcf, fullfile(plotDir, 'matlab_staircase.png'), 'Resolution
   ', 300);
%% dimension calculation


% finding flat segments
tol = 1e-5;
flat = [0, abs(diff(W)) < tol, 0];
starts = find(diff(flat) == 1);
```

```
ends   = find(diff(flat) == -1);
widths = Omega(ends) - Omega(starts);


r  = linspace(2e-3, 0.1, 101);
sr = arrayfun(@(rk) sum(widths(widths >= rk)), r); % iterating over r(max width) values
Nr = (1 - sr) ./ r; % amount of empty space

x = log(1./r);
y = log(Nr);
p = polyfit(x, y, 1);
fprintf('slope = %.3f', p(1));

figure;
plot(x, y, 'b.','DisplayName', '$N(r)$ from plateau widths');
hold on
plot(x, polyval(p, x), 'r-', 'LineWidth', 2, 'DisplayName', sprintf('$y$ = %.4f $x$ + %.4f', p(1), p(2)));
grid on;
xlabel('log 1/r');
ylabel('log N(r)');
legend('show', 'Location', 'best', 'Interpreter', 'latex');
exportgraphics(gcf, fullfile(plotDir, 'matlab_dimension_fit.png'), 'Resolution', 300);
```

B Python reproduction scripts

Listing 2: Python script for the chaos game and circle coverage.

```
"""Reproduce Barnsley and Vince, Example 1; export data for PGFPlots."""
from pathlib import Path
import numpy as np
import argparse

ROOT = Path(__file__).resolve().parent


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data-only", action="store_true", help="Export CSV without Matplotlib")
    args = parser.parse_args()
    for folder in ("data", "plots"):
        (ROOT / folder).mkdir(exist_ok=True)
    rng = np.random.default_rng(535)
    steps = rng.integers(0, 2, size=4096)
    phases = (np.cumsum(steps) * ((np.sqrt(5) - 1) / 2)) % 1
    points = np.column_stack((np.cos(2 * np.pi * phases), np.sin(2 * np.pi * phases)))
    sizes = 2 ** np.arange(3, 13)
    distances = []
    for n in sizes:
```

```
        ordered = np.sort(np.unique(phases[:n]))
        gap = np.diff(np.r_[ordered, ordered[0] + 1]).max()
        distances.append(2 * np.sin(np.pi * gap / 2))
    assert np.all(np.diff(distances) <= 1e-14) and distances[-1] < 0.01
    np.savetxt(ROOT / "data/circle.csv", points[:512], delimiter=",",
        header="x,y", comments="")
    np.savetxt(ROOT / "data/coverage.csv", np.c_[sizes, distances],
        delimiter=",",
               header="n,distance", comments="")
    print(f"Coverage errors at N=8 and N=4096: {distances[0]:.6g}, {
        distances[-1]:.6g}")
    if args.data_only:
        return
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    fig, axes = plt.subplots(1, 2, figsize=(8, 3.6), layout="constrained")
    axes[0].scatter(*points[:512].T, s=3)
    axes[0].set(aspect="equal", xlabel="x", ylabel="y", title="First 512
        iterates")
    axes[1].loglog(sizes, distances, "o-")
    axes[1].set(xlabel="Number of iterates", ylabel="Hausdorff distance",
        title="Circle coverage")
    fig.savefig(ROOT / "plots/chaos_game.png", dpi=180)


if __name__ == "__main__":
    main()
```

Listing 3: Python script for the winding number staircase.

```
"""Reproduce Bak's Figure 5c numerically and export data for PGFPlots."""
from pathlib import Path
import numpy as np
import argparse

ROOT = Path(__file__).resolve().parent


def winding(bias, burn, samples, coupling=1):
    phase = np.full_like(bias, 0.123)
    displacement = np.zeros_like(bias)
    for n in range(burn + samples):
        increment = bias + coupling / (2 * np.pi) * np.sin(2 * np.pi *
            phase)
        phase = (phase + increment) % 1
        if n >= burn:
            displacement += increment
    return displacement / samples


def main():
    parser = argparse.ArgumentParser(description=__doc__)
```

```
    parser.add_argument("--data-only", action="store_true", help="Export␣CSV␣without␣Matplotlib")
    args = parser.parse_args()
    for folder in ("data", "plots"):
        (ROOT / folder).mkdir(exist_ok=True)
    bias = np.linspace(0, 1, 3001)
    w = winding(bias, 2048, 8192)
    w_long = winding(bias, 4096, 16384)
    assert np.allclose(winding(bias[:10], 4, 16, coupling=0), bias[:10])
    assert np.max(np.abs(w - w_long)) < 0.001
    for target in (0, 1/3, 1/2, 2/3, 1):
        locked = bias[np.abs(w_long - target) <= 2e-5]
        assert len(locked) > 1
        print(f"W={target:.6g}:␣grid␣interval␣[{locked[0]:.6f},␣{locked[-1]:.6f}]")
    print(f"Maximum␣change␣under␣doubling:␣{np.max(np.abs(w␣-␣w_long)):.6g}")
    np.savetxt(ROOT / "data/staircase.csv", np.c_[bias, w_long], delimiter=",",
               header="bias,winding", comments="")
    if args.data_only:
        return
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    fig, ax = plt.subplots(figsize=(6, 4), layout="constrained")
    ax.plot(bias, w_long, color="tab:blue")
    ax.plot(bias, bias, "--", color="0.5", label="Uncoupled")
    ax.set(xlabel="Bias", ylabel="Winding␣number", title="Critical␣sine␣circle␣map")
    ax.legend()
    fig.savefig(ROOT / "plots/devil_staircase.png", dpi=180)


if __name__ == "__main__":
    main()
```